

Essential Linux commands

Basic command line operations

Nikolaj Lindberg
nikolaj.lindberg@ling.su.se

August 2026

Basic Linux commands

▸ Missing symbols

▸ shortcut keys

▸ completion

▸ exit

▸ path shortcuts

▸ redirect

▸ cat

output contents of a file

▸ cd

change directory

▸ cp

copy files and directories

▸ echo

output a single line of text

▸ less

show contents of a file (see more)

▸ ls

list directories and files

▸ man

manual page for a command

▸ mkdir

create a new directory

▸ more

show contents of a file (see less)

▸ mv

move and rename files

▸ pwd

print working directory (where you are)

▸ rm

remove files

▸ scp

copy files to/from a remote computer

▸ ssh

login to a remote computer

▸ wc

'word count'; counts lines, words and bytes

Additional Linux commands

▸ curl	download files from internet
▸ date	date and time
▸ egrep	string search
▸ head	view beginning of file
▸ hostname	name of computer
▸ micro	text editor
▸ rev	reverse text lines
▸ shuf	random lines from a file
▸ sort	sort lines
▸ tail	view end of file
▸ uname	system information, such as operating system (OS)
▸ uniq	remove adjacent duplicate lines
▸ wget	download files from internet
▸ which	locate a command (full path)
▸ who	show users logged in

Symbols missing from keyboards in the computer lab

\$	option 4			
	option 7			
\	option-shift 7			
[	option 8			
]	option 9			
{	option-shift 8			
}	option-shift 9			
~	option "			(to the right of )

Option  is called *alt* on some keyboards.

Terminal editing shortcuts

,  move cursor left and right
,  scroll up or down in command history

 +  jump to beginning of line

 +  jump to end of line

 +  switch places of two characters

 +  clean up terminal window

 +  search the command history, to reuse earlier commands

`clear` clean up terminal window (same as  + )

`cd -` jump back to previous directory

Tab completion: saves time, avoids typing mistakes

After typing a few characters of a command or file name, you can often use *tab completion* to fill in the rest:

```
ls /h
```

```
ls /home/
```

```
ls /home/d
```

```
ls /home/dsv/
```

If no unambiguous expansion, hit  twice for suggestions.

To list all commands beginning `h`:

```
h
```

Exit from commands: `ctrl` + `C`, `ctrl` + `D`, `Q`

When you want to abort or exit a command, try one of these:

`ctrl` + `C` Terminate command (e.g. when you are stuck and bewildered)

`ctrl` + `D` Stop a command waiting for input (sends EOF, end-of-file).

`ctrl` + `D` If the command line is empty: exit the terminal

`Q` Some interactive commands exit when you type `Q` (*quit*)

Shortcut paths: home; up; here; file path wildcard

- ~ The path to your home directory
- ~baal The path to user baal's home directory
- .. The directory above:
ls ..
cd ..
- . Current directory, "here", useful destination when copying files:
cp ~baal/todo.txt .
- * File name wildcard. List files in texts/ with extension .txt:
ls texts/*.txt

>, >>, | : Redirect to file; append to file; chain commands

- > Write to file:

```
echo "Save this text" > file.txt
```

(If file already exists, over-writes existing contents;
otherwise creates new file)

- >> Append to file:

```
echo "Append this line" >> file.txt
```

(Adds to contents, if file already exists. If not, creates new file.)

- | Combine commands (*pipeline*):

```
echo "How many words?" | wc -w
```

(wc reads its input from echo)

cat: Output contents of files

`cat ks1.txt` Print contents of file `ks1.txt`

`cat f1.txt f2.txt` Contents of `f1.txt` and `f2.txt`

`cat -n ks1.txt` Add line numbers to output

`cat` followed by `|` can be used for sending the contents of files to other commands.

Example: Send the contents of all files with extension `.txt` to `wc -l` (count lines):

```
cat *.txt | wc -l
```

Using the vertical bar, `|`, to connect commands is called a *pipeline*

cd change directory — Examples

cd 	go to your home directory
cd lab1	go to sub-directory lab1/ in current directory
cd lab1/	go to sub-directory lab1/ in current directory
cd /tmp	go to /tmp (directly under root, /)
cd ~kali2818/lab1	go to user kali2818's lab1/ directory
cd ..	go to directory above
cd ../..	go two directories above
cd -	return to previously visited directory

cp: copy files <from> <to> destination

```
cp f.txt f.txt.bak
```

Create copy of f.txt called f.txt.bak

```
cp texts/ksl.txt corpus/
```

Copy file ksl.txt from
directory texts/ into directory corpus/

```
cp texts/ksl.txt .
```

Copy file from sub-directory texts/
to current directory (.). Keep file name

```
cp texts/*.txt corpus/
```

Copy all files with extension .txt in
texts/ to corpus/

```
cp -r scripts/ lab1/
```

Copy directory scripts/ into lab1/
The switch -r ensures that everything
in scripts/ is copied (*recursively*)

echo: Print a single line of text

```
echo dog
```

Print the string dog to the terminal

```
echo dog > canine.txt
```

Create a file `canine.txt` with a single line of text

(Using the `>` operator to redirect output to a file)

```
echo dog | rev
```

Send the string dog to the `rev` command

(`|` chains two commands together; `rev` reverses its input)

```
echo $PATH
```

Print the value of the `$PATH` environment variable. (Try also e.g. `$RANDOM`, `$PWD` or `$HOME`)

less: View contents of text file (see more)

`less file.txt` View a file one screen-full at a time.
Space bar, or  and  to scroll.
 to quit

If some command produces a lot of output, you can send that output to `less`, and scroll through it:

```
egrep whale moby_dick.txt | less
```

(`egrep` is a command for searching text. Notice the vertical bar that connects two commands)

ls list — Examples

<code>ls</code>	contents of current directory
<code>ls /home/dsv/kali2818</code>	contents of the given the directory
<code>ls -a</code>	show hidden files (beginning with dot, .)
<code>ls -sh</code>	show file size
<code>ls -l</code>	show file info (NB: small letter L, not digit 1)
<code>ls *.py</code>	list files ending .py (in current dir)
<code>ls lab*.txt</code>	list files starting with lab and ending .txt
<code>ls ~</code>	contents of your home directory (~)
<code>ls ~kali2818</code>	contents of user kali2818's home directory

man: The manual page of a command

man Manual page for a command

man egrep The manual for the egrep command

Scroll using  and  or the space bar.
 to quit.

Useful for checking the switches (flags) of a command you are familiar with. May not be ideal for learning a new command.

mkdir: Create a new directory

<code>mkdir corpus</code>	Create a new directory called corpus
<code>mkdir corpus/books</code>	Create a sub-directory to corpus called books

more: View contents of text file (see less)

`more file.txt` View a file one screen-full at a time.
Space bar, or  and  to scroll.
 to quit

If some command produces a lot of output, you can send that output to `more`, and scroll through it:

```
egrep whale moby_dick.txt | more
```

(`egrep` is a command for searching text. Notice the vertical bar that connects two commands)

`mv`: Move and rename files and directories <from> <to>

```
mv f.txt file.txt
```

Rename file `f.txt`

```
mv script/ scripts
```

Rename directory `script/`

```
mv texts/ksl.txt corpus/
```

Move file `ksl.txt`
from directory `texts/` to `corpus/`

```
mv texts/ksl.txt poem.txt
```

Move and rename

pwd: Print working directory

pwd The path to the current directory

Example: User kali2818 issues the pwd command:

```
pwd
/home/dsv/kali2818
(kali2818 is currently in the home directory)
```

Change to sub-directory public_html, and repeat:

```
cd public_html/
pwd
/home/dsv/kali2818/public_html
```

rm: Remove files and directories (*forever*)

<code>rm ksl.txt</code>	Remove file <code>ksl.txt</code>
<code>rm scripts/*.py</code>	Remove all files in directory <code>scripts</code> with the extension <code>.py</code>
<code>rm -r scripts/</code>	Remove the <code>scripts</code> directory and all its contents
<code>rm -i poem.txt</code>	Ask before deleting (the user has to type <code>y</code> to confirm)

You cannot undo the `rm` command.

(One way to protect oneself from terrible accidents is to use a version control system, such as `git`)

scp Copy files between machines

scp is used to copy files to or from a remote computer.

You need to have a user account on that computer.

You will need to know your user name and your password.

```
scp <USER>@<HOSTNAME>:<FILE PATH> <LOCAL DESTINATION>  
scp <FILE TO COPY> <USER>@<HOSTNAME>:<REMOTE  
DESTINATION>
```

Example: from remote machine

```
scp kali2818@triton.dsv.su.se:texts/ksl.txt .
```

(Notice the dot at the end; cf. cp)

Example: to remote machine

```
scp memoir.txt kali2818@triton.dsv.su.se:texts/
```

ssh: Remote login

ssh is used to log in on a different computer.

You need to have a user account on that computer.

You will need to know your user name and your password.

When logging in to `triton.dsv.su.se`, your user name and password are your SUKAT-ID and password, issued by Stockholm University.

Example:

```
ssh kali2818@triton.dsv.su.se
```

Substitute `kali2818` with your own SUKAT-ID. You'll be prompted for your password.

wc: Count lines, words, bytes, characters

`wc ksl.txt` Reports number of lines, words and bytes in `ksl.txt`

`wc -l` Number of lines (NB: small letter L, not digit 1)

`wc -w` Number of words

`wc -c` Number of bytes

`wc -m` Number of characters

curl: Download files from the internet

`curl` Given a URL, download a file and print it to the terminal
(*standard output*)

`curl -L` Follow link to a file (redirect). Often needed,
depending on web-server configurations

Example: Download and print the file `ks1.txt` to the terminal:

```
curl -L people.dsv.su.se/~kali2818/ks1.txt
```

```
Över alla mödor
```

```
jag ror.
```

```
Står du på stranden
```

```
och gråter, min bror?
```

```
...
```

date: Report current time

<code>date</code>	Reports the current date and time according to the computer
<code>date +%s</code>	Prints the number of seconds since January 1 1970
<code>date +%A</code>	Prints the name of the current weekday

See `man date` for more formatting options.

egrep: Search for strings in text files

egrep uses regular expressions (search patterns) to match lines in text files.

Extract the lines containing the string `whale` from the file `moby_dick.txt`:

```
egrep whale moby_dick.txt
```

Extract the lines containing Captain Ahab or Captain Peleg from the file `moby_dick.txt`:

```
egrep 'Captain (Ahab|Peleg)' moby_dick.txt
```

head: Output a number of lines from the beginning of a file

<code>head moby_dick.txt</code>	Print first 10 lines of file <code>moby_dick.txt</code>
<code>head -n 67 moby_dick.txt</code>	Print first 67 lines of file <code>moby_dick.txt</code>

hostname: Print the name of the computer

<code>hostname</code>	The name of the computer, such as <code>triton</code>
<code>hostname -f</code>	Full domain name, such as <code>triton.dvs.su.se</code>

micro: A terminal text editor

`micro file.txt` Open file `file.txt` for editing

BASIC EDITING COMMANDS

ctrl + S	save
ctrl + Q	quit
ctrl + Z	undo
↑ + ↓ etc	mark text
ctrl + C	copy
ctrl + V	paste

rev: Reverse the characters of lines of text

<code>rev file.txt</code>	Reverse each line of the file <code>file.txt</code>
<code>echo pool mood rev</code>	Reverse the output of the <code>echo</code> command

shuf: Randomise (*shuffle*) lines of a file

<code>shuf file.txt</code>	Randomise the order of the lines of <code>file.txt</code>
<code>shuf -n 28 file.txt</code>	Select 28 random lines from <code>file.txt</code>

sort: Sort lines

<code>sort file.txt</code>	Sort the lines of <code>file.txt</code> alphabetically
<code>sort -r file.txt</code>	Sort the lines of <code>file.txt</code> in reversed order
<code>sort -n file.txt</code>	Sort the lines of <code>file.txt</code> numerically

tail: Output a number of lines from the end of a file

<code>tail moby_dick.txt</code>	Print last 10 lines of file <code>moby_dick.txt</code>
<code>tail -n 67 moby_dick.txt</code>	Print last 67 lines of file <code>moby_dick.txt</code>

uname: System information, such as operating system

<code>uname</code>	Name of operating system (kernel) of the computer
<code>uname -a</code>	Print full information (system version, etc)

uniq: Remove identical *adjacent* lines

uniq is used to remove identical lines that *directly follow* each other. This means that to use it to remove *all* duplicate lines, the lines must first be sorted.

```
uniq sorted.txt
```

Remove duplicate lines from file sorted.txt

```
uniq -c sorted.txt
```

Also *count* duplicate lines in sorted.txt

```
sort file.txt | uniq -c
```

Sort and report frequency of identical lines in file.txt

wget: Download files from internet

The command

```
wget people.dsv.su.se/~kali2818/ksl.txt
```

downloads the file `ksl.txt` and saves it to a file with the same name (`ksl.txt`) in the current directory.

which: The full path to a command

which is used to find out what file is actually executed when a command, such as `egrep`, is issued. Can be useful when there are competing commands with the same name in different directories.

```
which egrep  Print the path to the egrep command
```

who: List users logged in to the computer

who is used to list the user names of those currently logged in. The same user name may occur more than once, e.g. if a user has multiple connections.

A classic *pipeline*: Word frequencies

You are not supposed to understand this, but for reference, here's a classic use of Linux pipelines.

This command creates a word frequency list from the input text files:

```
cat *.txt | egrep -o '\w+' | sort | uniq -c | sort -rn
```

Are you able to pick that pipeline apart and figure out how it works...?